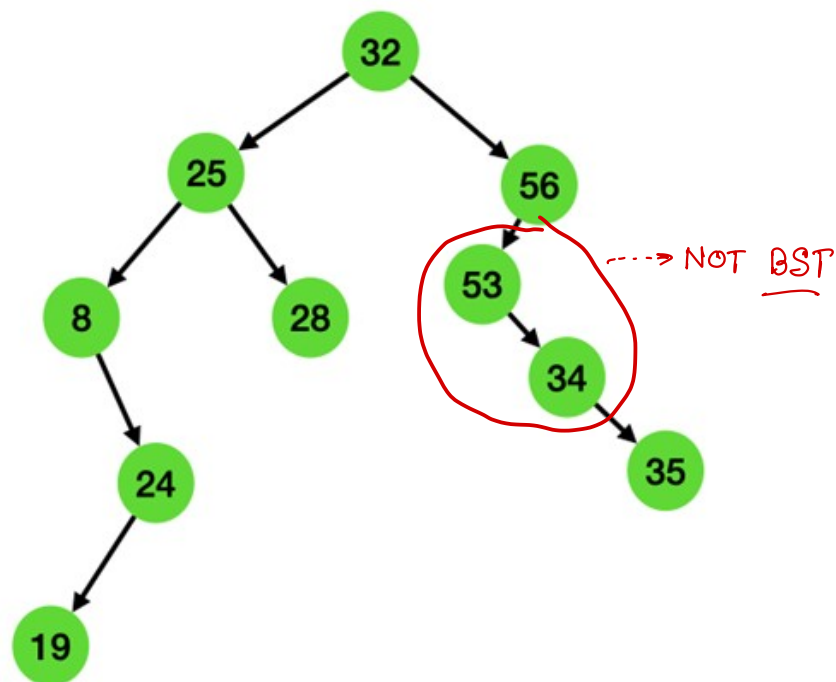


CPE 112 Programming with Data Structures

Exercises: Trees & Graphs

Trees

จากโครงสร้างต้นไม้ที่กำหนดให้ จงตอบคำถามต่อไปนี้



1. ต้นไม้เป็น Binary Tree, Binary Search Tree, หรือ AVL Tree

Answer: ต้นไม้ไม่เป็น Binary tree ครมต

2. ต้นไม้มีกี่ Level

Answer: 5 Levels

3. มี Internal Node ใดบ้างที่มีความสูงเท่ากัน

Answer: Height = 1 : {24, 34} Height = 3 : {25, 56}
Height = 2 : {8, 53}

4. 32 56 25 53 28 8 34 24 19 35 เป็นลำดับข้อมูลที่ถูกใส่เข้าไปใน โครงสร้างต้นไม้ใช่หรือไม่

Answer: ใช่ (Binary Tree เราสามารถใส่ลำดับแบบไหนก็ได้)

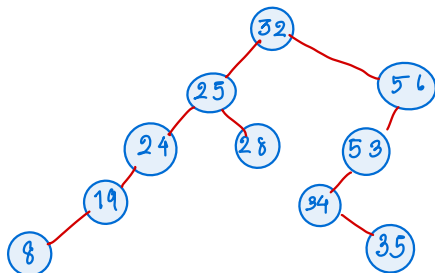
5. รูปแบบของ ลำดับข้อมูล ที่ถูก ใส่เข้าไป ใน โครงสร้าง ต้นไม้ สามารถ มี ได้ รูปแบบ เดียวหรือไม่ หากใช่ให้ทำข้อ

(a) อธิบายสาเหตุที่รูปแบบของ ลำดับข้อมูล มี ได้ เพียง รูปแบบ เดียว หาก ไม่ใช่ให้ข้ามไปทำข้อ 5b

Answer: -

(b) ยกตัวอย่าง ลำดับข้อมูลแบบอื่นเพิ่มเติม พร้อมทั้ง หาความสัมพันธ์ ระหว่าง ลำดับข้อมูลกับ โครงสร้าง ต้นไม้ ที่ ได้

Answer: 32, 56, 25, 53, 34, 35, 28, 24, 19, 8
เป็นต้นไม้แบบ Binary Search Tree



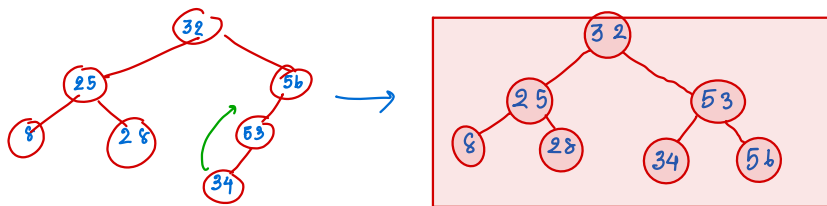
6. หากใช้ส่วนของโปรแกรม postOrder ด้านล่าง สำหรับทำ Post-order Traversal โดยส่ง Root Node เป็นพารามิเตอร์เข้าสู่ฟังก์ชัน โปรแกรมในบรรทัดที่ 6 จะถูกรันกี่ครั้ง

1	void <u>postOrder</u> (TREENODE_T* node)
2	{
3	if (node-> <u>left</u> != NULL)
4	<u>postOrder</u> (node->left)
5	if (node-> <u>right</u> != NULL)
6	<u>postOrder</u> (node->right)
7	<u>printf</u> ("Access %d\n", node->data);
8	}

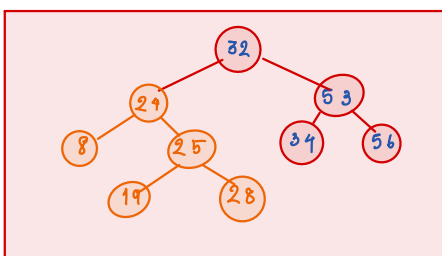
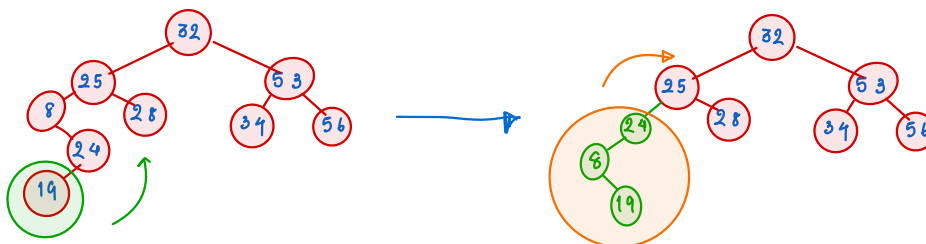
Answer: 5 ครั้ง

7. หากต้องใช้ลำดับข้อมูล 32 56 25 53 28 8 34 24 19 35 ในการสร้าง AVL Tree จะเกิดการเสียสมดุลในขั้นตอนที่เพิ่มโหนดใดบ้าง เป็นการเสียสมดุลแบบใด

Answer: ตอนเพิ่ม node 34
เสียสมดุลแบบ left ของ right-sub-tree

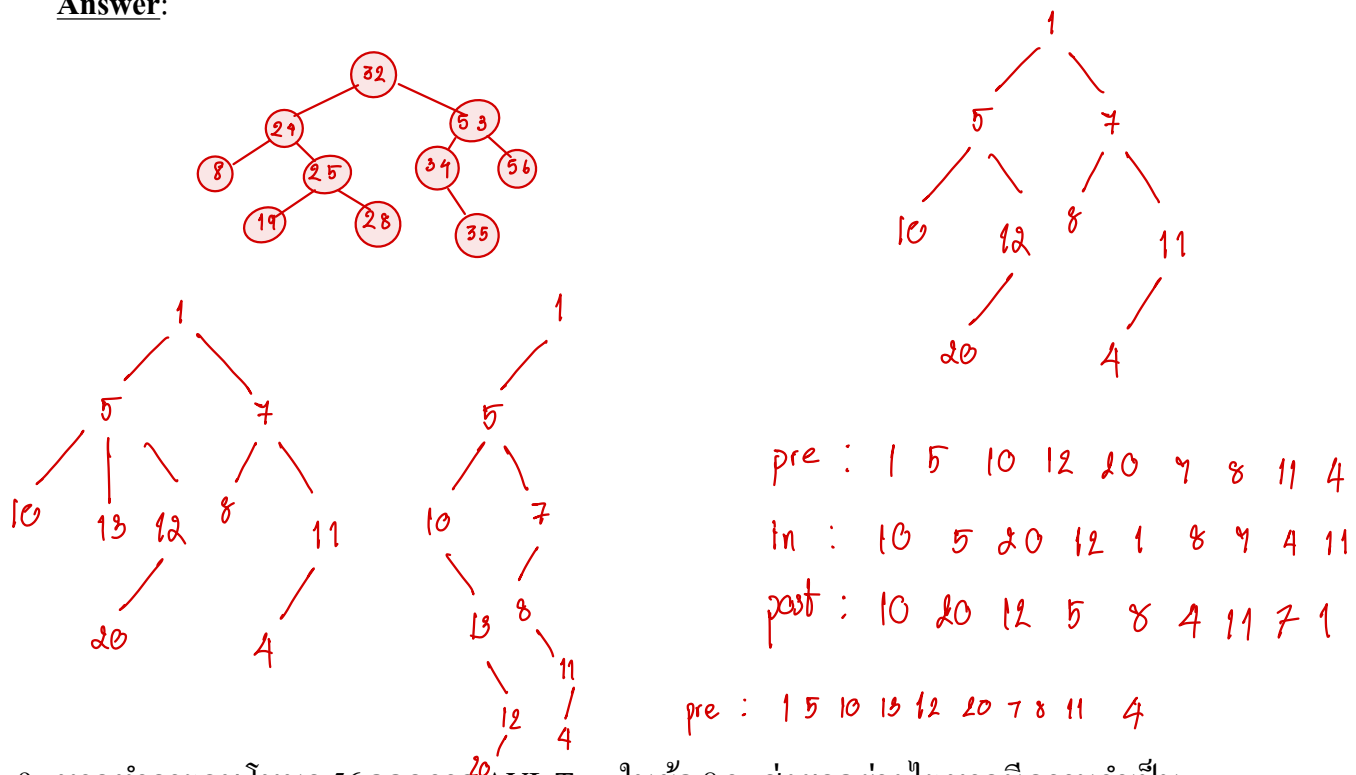


ตอนเพิ่ม node 19
เสียสมดุลแบบ left ของ left-sub-tree



8. ผลลัพธ์ AVL Tree ในข้อ 7 มีรูปแบบอย่างไร

Answer:



9. หากทำการลบ โหนด 56 ออกจาก AVL Tree ในข้อ 8 จะส่งผลอย่างไร หากมีความจำเป็น ต้องแก้ไขให้เป็นไปตามหลักการของ AVL Tree จะต้องดำเนินการอย่างไร

Answer:



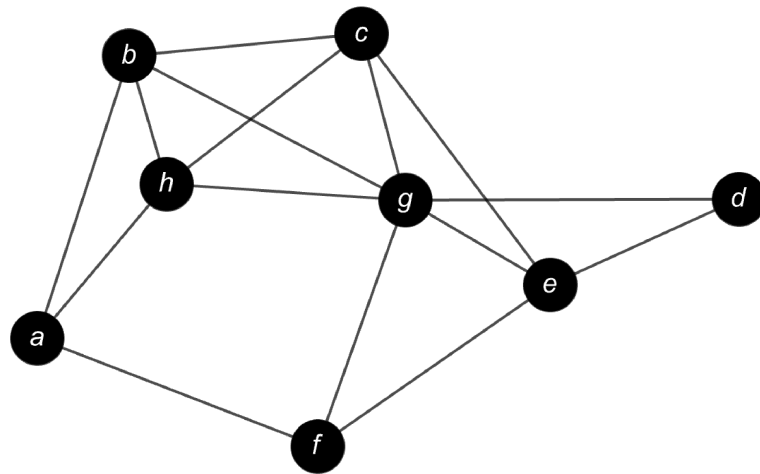
10. (Optional) จาก โครงสร้าง ข้อมูล ที่ กำหนด ให้ เติม ส่วน ของ โค้ด ภาษาซี ใน ฟังก์ชัน is-BST เพื่อ ตรวจสอบ ว่า Binary Tree ใดๆ เป็น Binary Search Tree หรือไม่ (หมายเหตุ: นศ.สามารถเขียนฟังก์ชันใหม่ทั้งหมดเองก็ได้)

```
typedef struct _treenode
{
    int data;
    struct _treenode* left;
    struct _treenode* right;
} TREENODE_T;

int isBST (TREENODE_T *root)
{
    if ( root->left->data > root->data ..... || root->right->data < root->data ..... )
        return 0; //not BST
    else if ( ..root->left->data < root->data && ..root->right->data > root->data .. )
        return 1; ///is BST
    else
        return .....isBST(root->left) && isBST(root->right);
}
```

Graphs

1. จากกราฟที่กำหนดให้ จงหา adjacency list และ adjacency matrix



Adjacency list

Vertex	Adjacency list
a	b, f, h
b	a, c, g, h
c	b, e, g, h
d	e, g
e	c, d, f, g
f	a, e, g
g	b, c, d, e, f, h
h	a, b, c, g

Adjacency Matrix

	a	b	c	d	e	f	g	h
a	0	1	0	0	0	1	0	1
b	1	0	1	0	0	0	1	1
c	0	1	0	0	1	0	1	1
d	0	0	0	0	1	0	1	0
e	0	0	1	1	0	1	1	0
f	1	0	0	0	1	0	1	0
g	0	1	1	1	1	1	0	1
h	1	1	1	0	0	0	1	0

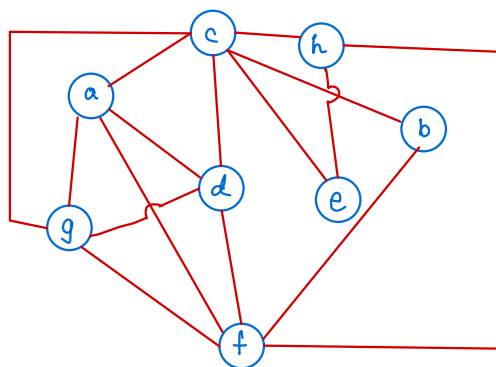
2. จาก adjacency list ที่กำหนดให้ จงหา graph และ adjacency matrix

Vertex	Adjacency vertex
a	c, d, f, g
b	c, f
c	a, b, d, e, g, h
d	a, c, f, g
e	c, h
f	a, b, d, g, h
g	a, c, d, f
h	c, e, f

Adjacency Matrix

	a	b	c	d	e	f	g	h
a	0	0	1	1	0	1	1	0
b	0	0	1	0	0	1	0	0
c	1	1	0	1	1	0	1	1
d	1	0	1	0	0	1	1	0
e	0	0	1	0	0	0	0	1
f	1	1	0	1	0	0	1	1
g	1	0	1	1	0	1	0	0
h	0	0	1	0	1	1	0	0

Graph



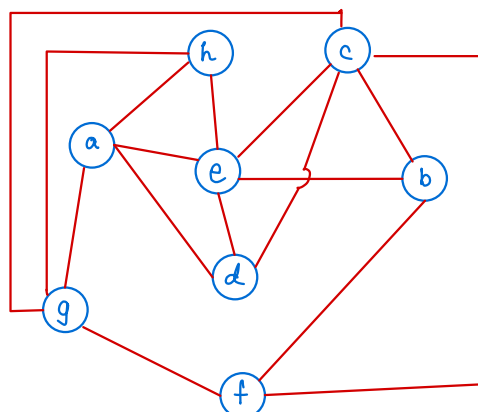
3. จาก adjacency matrix ที่กำหนดให้ จงหา graph และ adjacency list

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

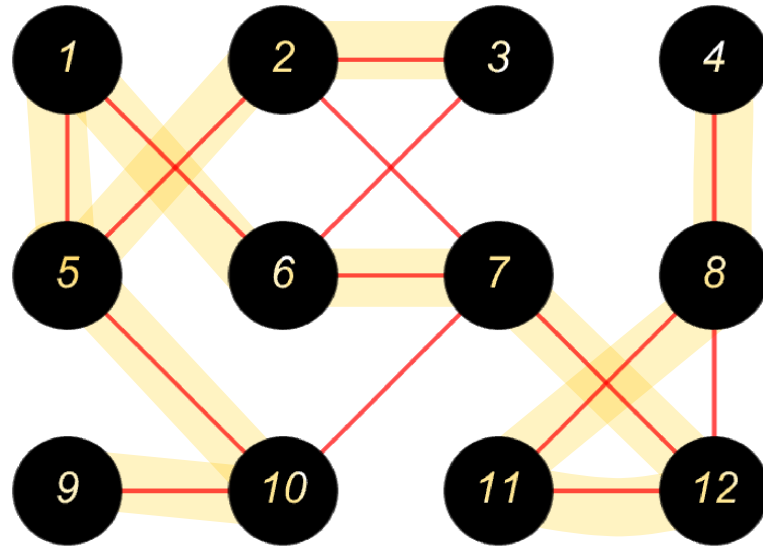
Adjacency list

Vertex	Adjacency list
a	d, e, g, h
b	c, e, f
c	b, d, e, f, g
d	a, c, e
e	a, b, c, d, h
f	b, c, g
g	a, c, f, h
h	a, e, g

Graph

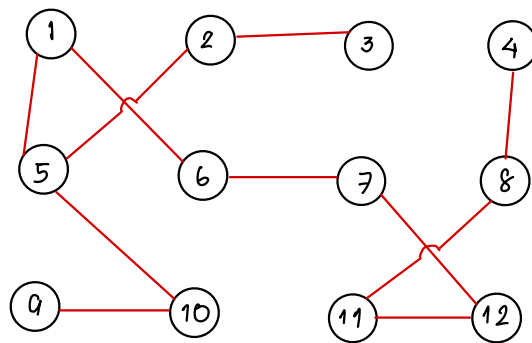


4. จงใช้ depth first search ในการหา graph traversal ของ graph ที่กำหนดให้ โดยให้เริ่มต้นที่ vertex 7 และให้ทำการ traversal ตามลำดับค่าใน vertex จากน้อยไปมาก

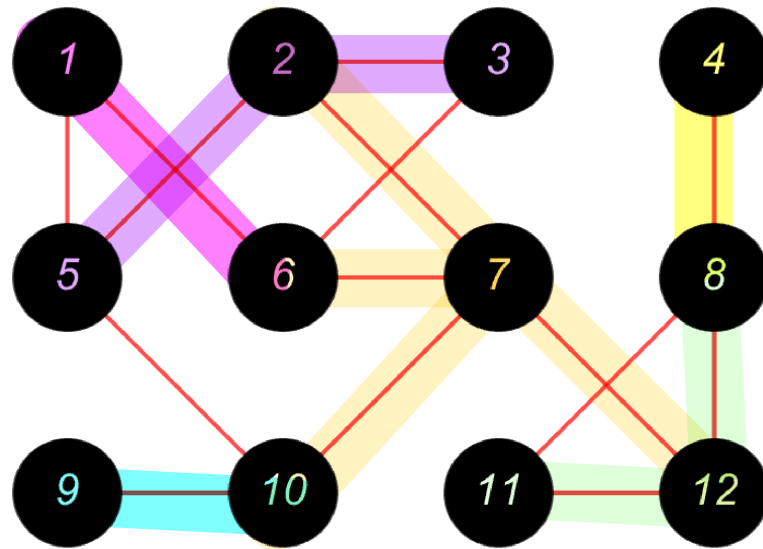


Stack list : 7, ~~6~~, ~~1~~, ~~5~~, ~~2~~, ~~3~~, ~~10~~, ~~9~~, 12, 11, 8, 4

ลำดับ : 7, 6, 1, 5, 2, 3, 10, 9, 12, 11, 8, 4



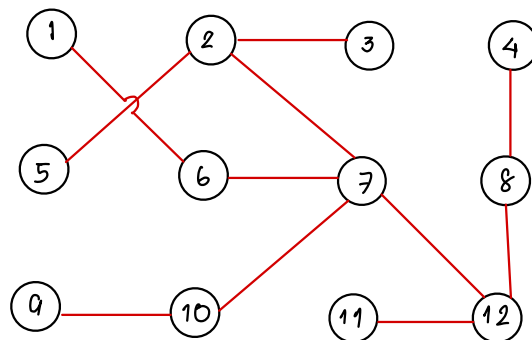
5. จงใช้ breadth first search ในการหา graph traversal ของ graph ที่กำหนดให้ โดยให้เริ่มต้นที่ vertex 7 และให้ทำการ traversal ตามลำดับค่าใน vertex จากน้อยไปมาก



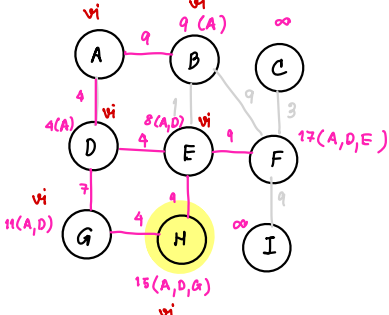
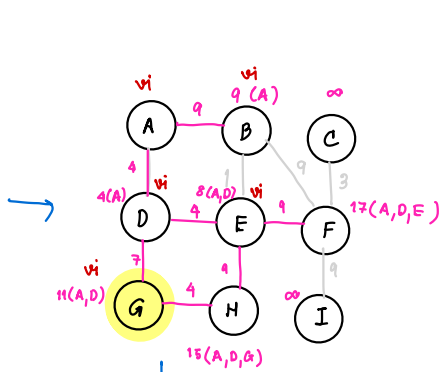
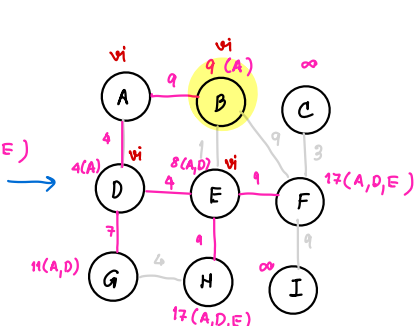
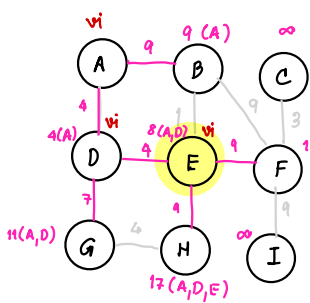
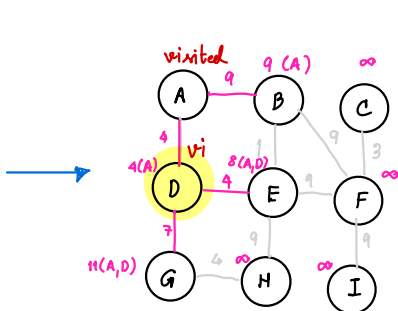
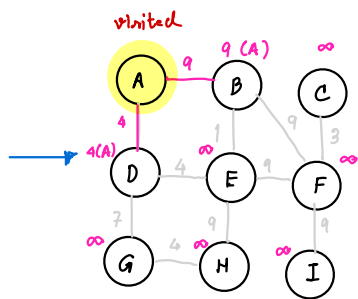
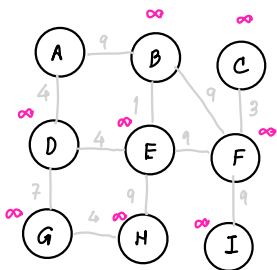
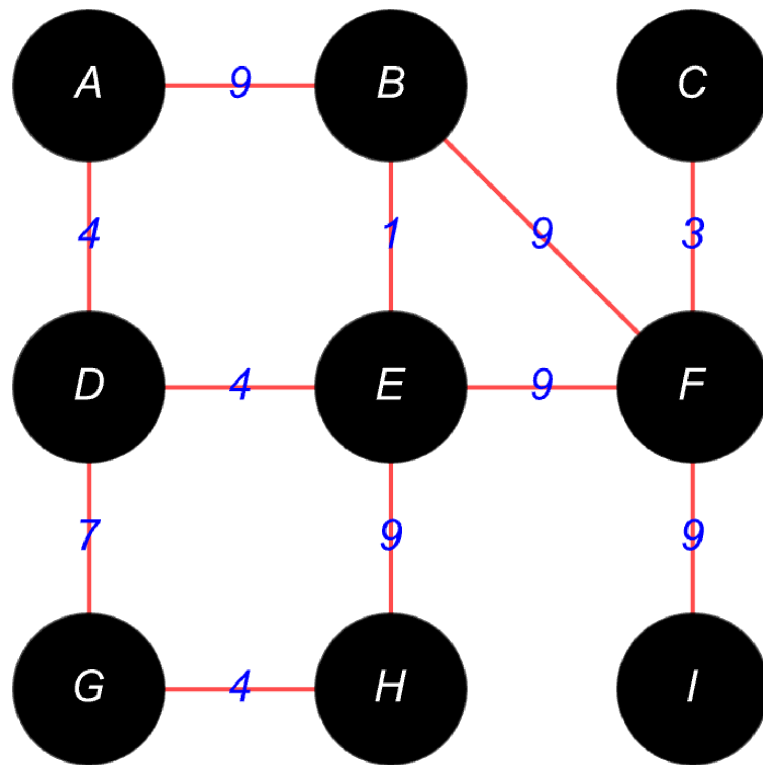
Queue list

~~1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12~~

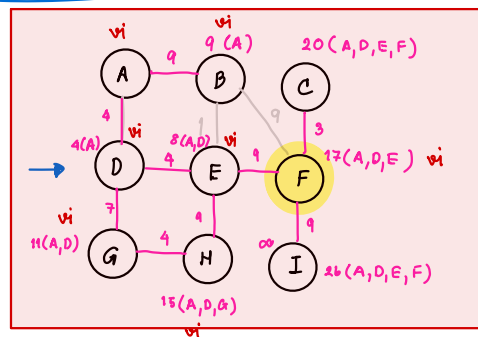
ลำดับ : 7, 2, 6, 10, 12, 3, 5, 1, 9, 8, 11, 4



6. จงใช้ Dijkstra's algorithm ในการหา shortest path จาก vertex ~~A~~^A ไปยัง vertex ต่าง ๆ

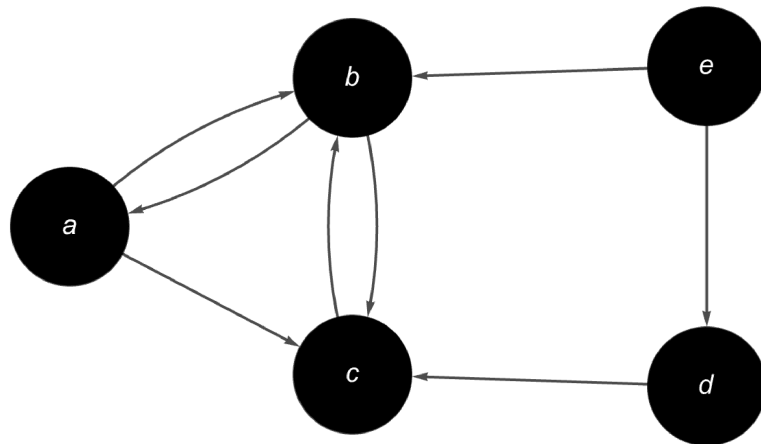


11



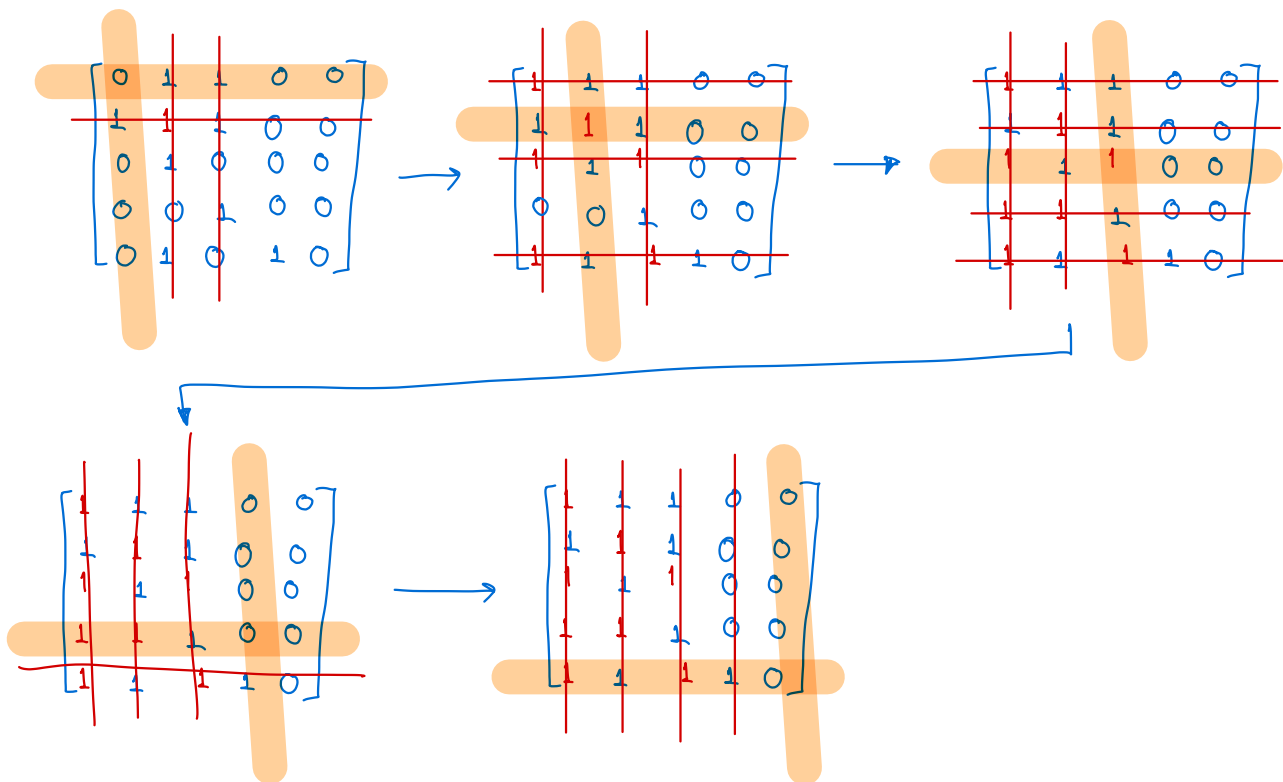
ตามแผนที่

7. จงใช้ Warshall's algorithm ในการหา transitive closure ของ graph ที่กำหนดให้



ทำ Graph ให้อ่านเป็น Adjacency Matrix ได้ดังนี้;

	a	b	c	d	e
a	0	1	1	0	0
b	1	0	1	0	0
c	0	1	0	0	0
d	0	0	1	0	0
e	0	1	0	1	0



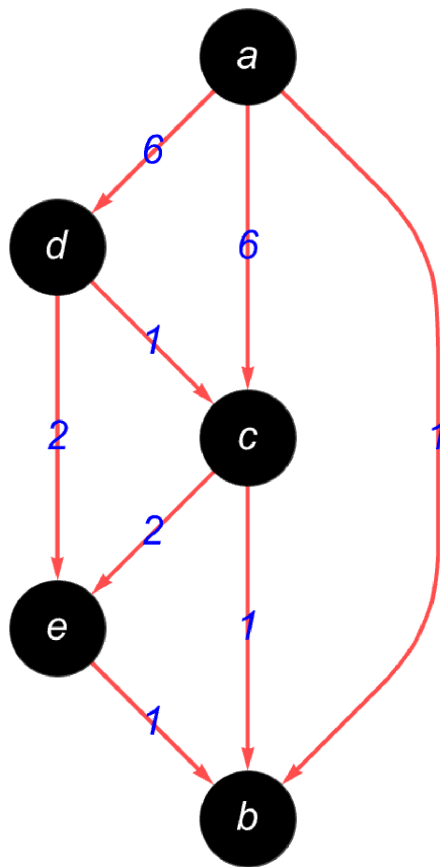
∴ Transitive Closure ของ Graph ดังกล่าว

1	1	1	0	0
1	1	1	0	0
1	1	1	0	0
1	1	1	0	0
1	1	1	1	0

8. จงใช้ Floyd's algorithm ในการหา all-pairs shortest path ของ graph ที่กำหนดให้

นำ Graph มาเขียน เป็น Matrix ได้ดังนี้

	a	b	c	d	e
a	0	1	6	6	∞
b	∞	0	∞	∞	∞
c	∞	1	0	∞	2
d	∞	∞	1	0	2
e	∞	1	∞	∞	0



0	1	6	6	∞
∞	0	∞	∞	∞
∞	1	0	∞	2
∞	∞	1	0	2
∞	1	∞	∞	0

0	1	6	6	∞
∞	0	∞	∞	∞
∞	1	0	∞	2
∞	∞	1	0	2
∞	1	∞	∞	0



0	1	6	6	8
∞	0	∞	∞	∞
∞	1	0	∞	2
∞	2	1	0	2
∞	1	∞	∞	0



0	1	6	6	8
∞	0	∞	∞	∞
∞	1	0	∞	2
∞	2	1	0	2
∞	1	∞	∞	0



0	1	6	6	8
∞	0	∞	∞	∞
∞	1	0	∞	2
∞	2	1	0	2
∞	1	∞	∞	0

$\therefore$ All pair shortest path ของ Graph นี้

0	1	6	6	8
∞	0	∞	∞	∞
∞	1	0	∞	2
∞	2	1	0	2
∞	1	∞	∞	0

